VOLUME 4 OF 12

# IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

## MODULE 4

ARRAYS, STRINGS & BASIC DATA HANDLING

*Authored by:*

***Mithila Rani Saha & Dithsa Dutta***

www.itstudyhub.dpdns.org

# Welcome to Module 4

## About This Module

Welcome to Volume 4 of the IT Study Hub Complete Java Programming Series. So far, you've managed single pieces of data. If you needed to store a student's grade, you created an `int`. But what happens when you need to store the grades of 1,000 students? Creating 1,000 separate variables is impossible to manage.

In this module, we transition from handling single elements to managing massive collections of data using **Arrays**. We will also dive deeply into the most important object in Java: the **String**, uncovering how Java handles text securely and efficiently.

## Learning Outcomes

- Understand the memory structure, declaration, and traversal of 1D and 2D Arrays.
- Implement basic searching (Linear) and sorting (Bubble) algorithms on array data.
- Master the `String` class, understanding the concept of Immutability and the String Pool.
- Perform complex text manipulation using built-in String methods.
- Optimize string processing using `StringBuilder` and `StringBuffer`.
- Bridge the gap between primitives and objects using Wrapper Classes, Autoboxing, and Unboxing.

## Module Coverage

We begin with One-dimensional Arrays and Multidimensional Arrays. We'll write loops to traverse, search, and sort these structures. Next, we dissect the `String` class, exploring concatenation, comparisons, and character handling. We will then

examine memory-efficient alternatives (StringBuilder) and wrap primitive data using Wrapper classes. Finally, we'll build a "Student Marks Analysis System" integrating all these concepts.

IT STUDY HUB

# 1. One-Dimensional Arrays

An array is a collection of variables of the **same data type** stored in contiguous (side-by-step) memory locations. Instead of declaring individual variables, you declare one array variable that holds multiple values.

### Array Memory Structure Concept

```
int[] scores = { 85, 92, 78, 90, 88 };
```




Index 0 Index 1 Index 2 Index 3 Index 4

*Length of array is 5. Indices always start at 0 and end at length - 1.*

IT STUDY HUB

## Concept: Array Declaration & Initialization

> **Simple Explanation:**
> Think of an array as an egg carton. You must tell Java exactly how many slots the carton has when you buy it. Once created, the size of an array can never be changed. You can either create an empty carton and fill it later, or buy a carton that comes pre-filled.

**Syntax / Structure:**

```
// Method 1: Declare size, fill later
dataType[] arrayName = new dataType[size];
arrayName[0] = value1;

// Method 2: Declare and initialize immediately
dataType[] arrayName = {value1, value2, value3};
```

**Example Code:**

```
public class ArrayBasics {
    public static void main(String[] args) {
        // Method 1: Create empty array of size 3
        String[] fruits = new String[3];
        fruits[0] = "Apple";
        fruits[1] = "Banana";
        fruits[2] = "Mango";

        // Method 2: Direct initialization
        int[] luckyNumbers = { 7, 13, 21 };

        // Accessing elements
        System.out.println("First fruit: " + fruits[0]);
        System.out.println("Second lucky number: " + luckyNumbers[1]);

        // Finding length
        System.out.println("Total fruits: " + fruits.length);
    }
}
```

**Expected Output:**

```
First fruit: Apple
Second lucky number: 13
Total fruits: 3
```

**Code Explanation:**
We access array slots using square brackets `[]` and the index number. Notice we use `fruits.length` (without parentheses) to find out how many slots the array has.

> ***Real-world Example:***
> *Storing the days of the week (`String[] days = {"Mon", "Tue"...};`) or the monthly temperatures for a year.*

> **Common Mistakes:**
>
> - Trying to access `fruits[3]` in an array of size 3. Since counting starts at 0, the valid indices are 0, 1, 2. Accessing 3 causes a massive error: `ArrayIndexOutOfBoundsException`!

> **Best Practices:**
>
> - Always use `arrayName.length` instead of hardcoding numbers when checking array sizes.

**Practice Task:**
**Create a double array of size 4 to store product prices. Assign values to all indices and print the 3rd price.**

---

## Concept: Array Traversal (Sum and Average)

> **Simple Explanation:**
> Instead of manually writing `print(array[0])`, `print(array[1])`, we use a `for` loop. The loop's counter variable `i` acts as the index, ticking up from 0 to the end of the array.

**Syntax / Structure:**

```
for (int i = 0; i < array.length; i++) {
    // Access array[i]
}
```

**Example Code:**

```
public class ArrayTraversal {
    public static void main(String[] args) {
        int[] marks = { 85, 90, 75, 80, 95 };
        int sum = 0;

        // Traverse the array
        for (int i = 0; i < marks.length; i++) {
            sum += marks[i]; // Add each mark to sum
            System.out.println("Mark " + (i+1) + ": " + marks[i]);
        }

        double average = (double) sum / marks.length; // Cast for decimal precision

        System.out.println("---");
        System.out.println("Total Sum: " + sum);
        System.out.println("Average: " + average);
    }
}
```

**Expected Output:**

```
Mark 1: 85
Mark 2: 90
Mark 3: 75
Mark 4: 80
Mark 5: 95
---
```

```
Total Sum: 425
Average: 85.0
```

**Code Explanation:**
The loop starts at `i = 0`. As long as `i < 5` (`marks.length`), it grabs `marks[i]` and adds it to `sum`. By the end of the loop, all 5 numbers are successfully summed.

***Real-world Example:***
*Calculating the total checkout cart price by traversing an array of item prices.*

**Common Mistakes:**

- Writing `i <= array.length` in the loop condition. Because indices end at length - 1, the `<= ` will cause an out-of-bounds crash. Always use strictly `<`.

**Best Practices:**

- For simple read-only traversal, modern Java prefers the 'Enhanced For Loop' (e.g., `for(int mark : marks) { ... }`).

**Practice Task:**
**Create an array of 5 integers. Use a loop to print ONLY the even numbers in the array.**

---

# 2. Multidimensional Arrays

### 2D Array (Matrix/Table) Concept

```
int[][] matrix = new int[3][4]; // 3 rows, 4 columns
```

| | Col 0 | Col 1 | Col 2 | Col 3 |
|---|---|---|---|---|
| **Row 0** | [0][0] | [0][1] | [0][2] | [0][3] |
| **Row 1** | [1][0] | [1][1] | [1][2] | [1][3] |
| **Row 2** | [2][0] | [2][1] | [2][2] | [2][3] |

IT STUDY

## Concept: 2D Arrays & Matrix Addition

> **Simple Explanation:**
> A 1D array is a single row of eggs. A 2D array is an entire pallet containing multiple rows and columns of eggs. You need two sets of brackets: `[row][column]`.

**Syntax / Structure:**

```
dataType[][] arrayName = new dataType[rows][cols];
// Accessing element at row 1, col 2:
arrayName[1][2] = 50;
```

**Example Code:**

```
public class MatrixAddition {
    public static void main(String[] args) {
        // Initialize two 2x2 matrices
        int[][] matrixA = { {1, 2}, {3, 4} };
        int[][] matrixB = { {5, 6}, {7, 8} };
        int[][] result = new int[2][2]; // Empty matrix to hold result

        System.out.println("Resulting Matrix:");
        // Outer loop for rows
        for (int i = 0; i < 2; i++) {
            // Inner loop for columns
            for (int j = 0; j < 2; j++) {
                result[i][j] = matrixA[i][j] + matrixB[i][j];
                System.out.print(result[i][j] + " ");
            }
            System.out.println(); // New line after each row
        }
    }
}
```

**Expected Output:**

```
Resulting Matrix:
6 8
10 12
```

**Code Explanation:**
We use a nested loop. The outer loop `i` selects the row. The inner loop `j` selects the specific column in that row. We add the elements at `[0][0]` together, then `[0][1]`, then move to the next row.

***Real-world Example:***
*Building a chessboard (8x8 2D array), plotting coordinates on a map, or representing pixels on a screen.*

**Common Mistakes:**

- Reversing the row and column coordinates. Always remember: `[row]` comes first, `[col]` comes second.

**Best Practices:**

- When traversing 2D arrays, use `matrix.length` for the number of rows, and `matrix[i].length` for the number of columns.

**Practice Task:**
**Create a 3x3 integer array. Fill it with numbers 1-9. Print the element exactly in the center (row 1, col 1).**

---

# 3. Basic Array Searching & Sorting

IT STUDY HUB

## Concept: Linear Search (Finding an element)

> **Simple Explanation:**
> Imagine looking for a specific book on a shelf. You start at the left, check the first book, check the second, and so on until you find it. If you reach the end and haven't found it, the book isn't there.

**Example Code:**

```
public class LinearSearch {
    public static void main(String[] args) {
        int[] ids = { 101, 205, 399, 412, 500 };
        int target = 399;
        boolean found = false;

        for (int i = 0; i < ids.length; i++) {
            if (ids[i] == target) {
                System.out.println("Target " + target + " found at index " + i);
                found = true;
                break; // Stop searching once found
            }
        }

        if (!found) { // equivalent to if(found == false)
            System.out.println("Target not found in array.");
        }
    }
}
```

**Expected Output:**

```
Target 399 found at index 2
```

**Code Explanation:**
We set a boolean flag `found` to `false`. We loop through the array. If `ids[i]` matches our target, we print the index, switch the flag to `true`, and `break` the loop immediately to save time.

> ***Real-world Example:***
> *Checking if a username already exists in a database of registered users.*

**Common Mistakes:**

- Forgetting to use a `break` statement after finding the element, causing the program to waste time searching the rest of the array needlessly.

**Best Practices:**

- For massive, sorted arrays, we use 'Binary Search' (advanced) instead of Linear Search, because it is exponentially faster.

**Practice Task:**
**Create an array of 5 names. Ask the user to type a name. Use Linear Search to print whether that name is in the array.**

IT STUDY HUB

## Concept: Finding the Largest/Smallest Element

> **Simple Explanation:**
> Assume the first number in the array is the 'King'. You walk down the line of numbers. If you find a number bigger than the current King, that new number becomes the King. At the end of the line, whoever is King is the largest number.

**Example Code:**

```
public class FindMinMax {
    public static void main(String[] args) {
        int[] numbers = { 45, 12, 89, 33, 7 };

        // Assume the first element is both min and max
        int max = numbers[0];
        int min = numbers[0];

        for (int i = 1; i < numbers.length; i++) {
            if (numbers[i] > max) {
                max = numbers[i]; // Found a new maximum
            }
            if (numbers[i] < min) {
                min = numbers[i]; // Found a new minimum
            }
        }

        System.out.println("Largest: " + max);
        System.out.println("Smallest: " + min);
    }
}
```

**Expected Output:**

```
Largest: 89
Smallest: 7
```

**Code Explanation:**
We initialize `min` and `max` using the value at index 0. We start our loop at `i = 1` (since we already checked index 0). The `if` statements constantly update `max` and `min` if better candidates are found.

***Real-world Example:***
*Finding the highest scoring student in a class, or the cheapest product in an online store.*

**Common Mistakes:**

- Initializing `max` to 0. This works for positive numbers, but if the array only contains negative numbers (e.g., `-5, -10`), 0 will remain the max, which is incorrect! Always initialize using `array[0]`.

**Best Practices:**

- Initialize tracking variables with `array[0]` to ensure logic works perfectly for all ranges of numbers.

---

IT STUDY H

# 4. The String Class

In Java, `String` is NOT a primitive data type (like `int` or `char`). It is an **Object** belonging to the `java.lang.String` class. It represents a sequence of characters.

IT STUDY HUB

## Concept: String Creation and Concatenation

> **Simple Explanation:**
> You can create Strings easily using double quotes (String Literal). To join two pieces of text together, we use the `+` operator, which is called 'Concatenation'.

**Syntax / Structure:**

```
// Method 1: String Literal (Preferred)
String s1 = "Hello";

// Method 2: using 'new' keyword
String s2 = new String("World");

// Concatenation
String result = s1 + " " + s2;
```

**Example Code:**

```
public class StringBasics {
    public static void main(String[] args) {
        String firstName = "John";
        String lastName = "Doe";

        // Concatenating strings
        String fullName = firstName + " " + lastName;

        // Concatenating string with numbers
        int age = 25;
        String profile = fullName + " is " + age + " years old.";

        System.out.println(profile);
    }
}
```

**Expected Output:**

```
John Doe is 25 years old.
```

**Code Explanation:**
When we use `+` between a String and an `int`, Java automatically converts the `int` into a String and glues them together.

***Real-world Example:***
*Generating dynamic emails: `"Dear " + userName + ", your order is confirmed!"`*

**Common Mistakes:**

- Using single quotes `'` for Strings. Single quotes are strictly for primitive `char` types.

**Best Practices:**

- Always create Strings using string literals (Method 1) instead of the `new` keyword to save memory.

**Practice Task:**
**Create strings for your city and country. Concatenate them with a comma in between and print.**

---

## Concept: String Immutability and Comparison

> **Simple Explanation:**
> When you create a String 'Apple', it's locked in memory forever. If you try to change it to 'Apple Pie', Java doesn't edit the original text. It creates a brand new String 'Apple Pie' somewhere else in memory. This is called Immutability.

**Syntax / Structure:**

```
// NEVER DO THIS FOR STRINGS:
if (str1 == str2)

// ALWAYS DO THIS:
if (str1.equals(str2))
```

**Example Code:**

```
public class StringCompare {
    public static void main(String[] args) {
        String literal1 = "Java";
        String literal2 = "Java";
        String objectStr = new String("Java");

        // .equals() checks the ACTUAL TEXT inside
        System.out.println("Text Match? " + literal1.equals(objectStr));

        // == checks if they are the exact same memory box
        System.out.println("Memory Match? " + (literal1 == objectStr));
    }
}
```

**Expected Output:**

```
Text Match? true
Memory Match? false
```

**Code Explanation:**
`.equals()` checks if the characters spell the same word. The `==` operator checks if they point to the exact same physical location in RAM. Because `objectStr` was created with `new`, it forced a new memory box to be made, so `==` fails.

***Real-world Example:***
*Verifying a user's typed password against the database password. You MUST use `.equals()`, or logins will randomly fail!*

**Common Mistakes:**

- Using `==` to compare Strings. This is a critical bug that ruins enterprise applications!

**Best Practices:**

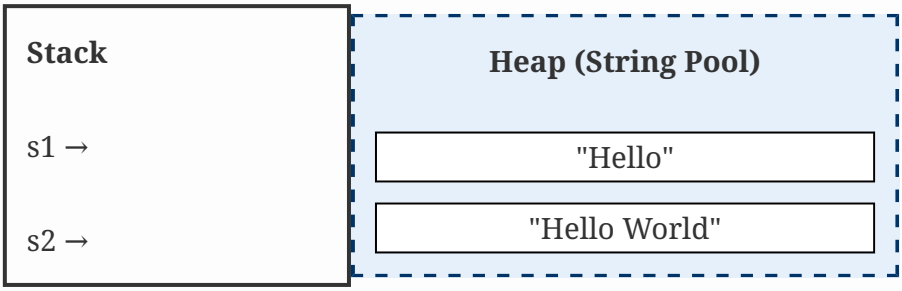
- Always use `.equals()` for text comparison. Use `.equalsIgnoreCase()` if you don't care about capital letters (e.g., 'yes' equals 'YES').

**Practice Task:**
**Take two string variables holding 'hello' and 'HELLO'. Compare them using .equalsIgnoreCase() and print the boolean result.**

---

## String Immutability & Memory Pool Concept

String literals are stored in a special memory area called the **String Pool**.




1. `String s1 = "Hello";` (Creates "Hello" in pool, s1 points to it).
2. `s1 = s1 + " World";` (Does NOT alter original "Hello". Creates NEW "Hello World" in pool, s1 now points to the new one. The original "Hello" stays untouched!).

## Concept: Useful Built-in String Methods

> **Simple Explanation:**
> You don't need to write complex loops to capitalize a word, find its length, or extract a specific letter. Java did the hard work for you.

**Example Code:**

```
public class StringMethods {
    public static void main(String[] args) {
        String text = "  IT Study Hub  ";

        // 1. Trim (removes surrounding spaces)
        String clean = text.trim();
        System.out.println("Trimmed: '" + clean + "'");

        // 2. Length (Count characters)
        System.out.println("Length: " + clean.length());

        // 3. ToUpperCase
        System.out.println("Caps: " + clean.toUpperCase());

        // 4. charAt (Get letter at specific index)
        System.out.println("First letter: " + clean.charAt(0));

        // 5. Substring (Extract a piece: start index, end index)
        System.out.println("Extracted: " + clean.substring(3, 8));
    }
}
```

**Expected Output:**

```
Trimmed: 'IT Study Hub'
Length: 12
Caps: IT STUDY HUB
First letter: I
Extracted: Study
```

**Code Explanation:**

`.trim()` deletes accidental spaces users type. `.length()` is a method (requires `()`), unlike

array length. `.substring(3, 8)` grabs characters starting from index 3 up to (but NOT including) index 8.

***Real-world Example:***
*`.trim()` is used on every single login form to ensure accidental spaces don't ruin passwords. `.substring()` is used to extract the area code from a phone number.*

**Common Mistakes:**

- Trying to use `.charAt()` with a number higher than length-1, causing a `StringIndexOutOfBoundsException`.

**Best Practices:**

- Chain methods safely to keep code clean: `String cleanName = name.trim().toUpperCase();`

**Practice Task:**
**Create a String 'Programming'. Use substring to extract 'gram' and print it.**

# 5. StringBuilder and StringBuffer

IT STUDY HUB

## Concept: Mutable Strings (StringBuilder)

> **Simple Explanation:**
> Because regular Strings are immutable (locked), adding text inside a loop creates thousands of abandoned strings in memory, slowing down the computer. `StringBuilder` is an editable whiteboard. You can erase and append text endlessly without wasting memory.

**Syntax / Structure:**

```
StringBuilder sb = new StringBuilder("Initial Text");
sb.append(" More Text");
```

**Example Code:**

```
public class StringBuilderExample {
    public static void main(String[] args) {
        // Create an editable string
        StringBuilder sb = new StringBuilder("Java");

        // Append (add to end)
        sb.append(" Programming");

        // Insert (add at specific index)
        sb.insert(5, "Awesome ");

        // Reverse the text completely
        sb.reverse();

        System.out.println(sb.toString()); // Convert back to normal String
    }
}
```

**Expected Output:**

```
gnimmargorP emosewA avaJ
```

**Code Explanation:**
Notice we didn't have to write `sb = sb.append()`. The `StringBuilder` actually edits itself in place. `.reverse()` is an incredibly powerful built-in method that standard Strings don't have!

***Real-world Example:***
*Assembling a massive HTML document or parsing a gigabyte XML file line-by-line.*

**Common Mistakes:**

- Using a regular String `+=` inside a loop that runs 10,000 times. It will crash or lag severely. Always use StringBuilder in loops!

**Best Practices:**

- Use `StringBuilder` for single-thread programs (most apps). Use `StringBuffer` ONLY if you are doing multi-threading (advanced).

---

## String vs StringBuilder vs StringBuffer

| Feature | String | StringBuilder | StringBuffer |
|---|---|---|---|
| **Mutability** | Immutable (Cannot be changed) | Mutable (Editable) | Mutable (Editable) |
| **Performance** | Fast (for read-only text) | **Very Fast** (for edits) | Slow (due to thread locks) |
| **Thread-Safe?** | Yes | No (Best for general use) | Yes (For multi-threading) |

---

# 6. Character Handling and Wrapper Classes

IT STUDY HUB

## Concept: Wrapper Classes, Autoboxing and Unboxing

> **Simple Explanation:**
> Java is an Object-Oriented language, but primitives (like `int`, `double`) are NOT objects. Some advanced Java tools (like ArrayLists) REFUSE to work with primitives. Wrapper classes are 'Object boxes' that we place primitive variables inside so they can be treated as objects. Java does this automatically (Autoboxing).

**Syntax / Structure:**

```
// Primitive
int a = 5;

// Wrapper Object
Integer b = 5; // Autoboxing (Automatic conversion)
```

**Example Code:**

```
public class WrapperExample {
    public static void main(String[] args) {
        // Autoboxing: Java automatically converts primitive int to Integer Object
        Integer objInt = 100;

        // Unboxing: Java automatically converts Integer Object back to primitive
        int primInt = objInt;

        System.out.println("Object Integer: " + objInt);
        System.out.println("Primitive int: " + primInt);

        // Wrapper classes have built-in helper methods!
        String numberStr = "250";
        int parsedNumber = Integer.parseInt(numberStr); // Convert text to math number
        System.out.println("Parsed Math Number: " + (parsedNumber + 50));
    }
}
```

**Expected Output:**

```
Object Integer: 100
Primitive int: 100
Parsed Math Number: 300
```

**Code Explanation:**
`Integer.parseInt()` is the most common use of Wrapper classes for beginners. It looks at a String containing numbers (e.g., from a website form), and safely converts it into a real primitive `int` so you can do math on it.

***Real-world Example:***
*Taking a zip code typed by a user in a web form (which is received as a String) and converting it to an `int` for database storage using `Integer.parseInt()`.*

**Common Mistakes:**

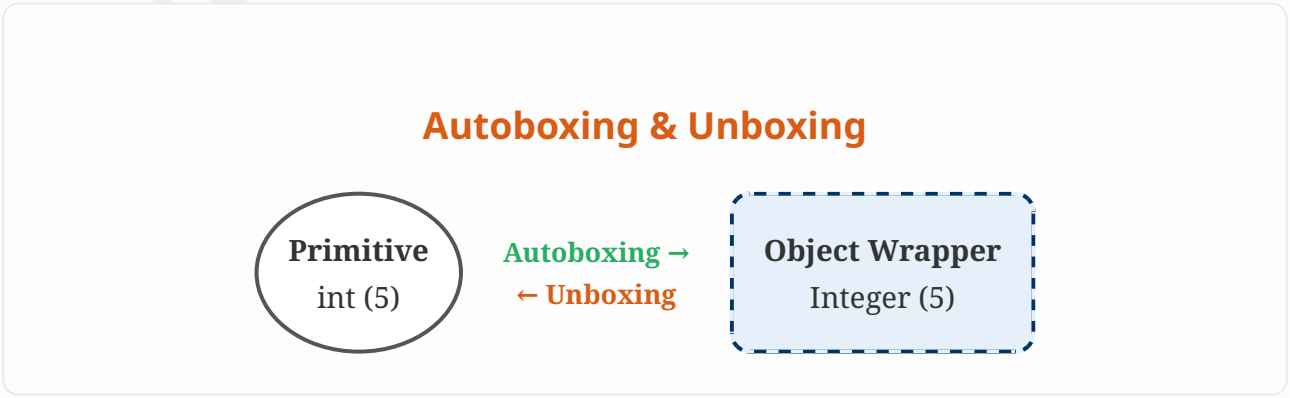
- Trying to run `.parseInt()` on a string that contains letters (e.g., `"250A"`). This throws a fatal `NumberFormatException`.

**Best Practices:**

- Learn the Wrappers: `int` -> `Integer`, `double` -> `Double`, `char` -> `Character`, `boolean` -> `Boolean` (Capitalize the first letter!).

**Practice Task:**
**Create a String holding the text '500'. Parse it into a primitive double using Double.parseDouble() and print it.**

---




---

# 7. Project: Student Marks Analysis System

Let's synthesize Arrays, Loops, and logic. We will build a console application that takes the marks of several students, stores them in an array, performs analytics (Total, Average, Max, Min), and allows the teacher to search for a specific grade.

```
import java.util.Scanner;

public class StudentMarksSystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("=== STUDENT MARKS ANALYZER ===");
        System.out.print("How many students are in the class? ");
        int numStudents = scanner.nextInt();

        // 1. Array Declaration (Dynamic size based on input)
        int[] marks = new int[numStudents];

        // 2. Data Entry (Initialization via Loop)
        System.out.println("\n--- ENTER MARKS ---");
        for (int i = 0; i < marks.length; i++) {
            System.out.print("Enter marks for Student " + (i + 1) + ": ");
            marks[i] = scanner.nextInt();
        }

        // 3. Analytics Variables
        int total = 0;
        int max = marks[0];
        int min = marks[0];

        // 4. Array Traversal & Logic
        for (int i = 0; i < marks.length; i++) {
            total += marks[i]; // Summation

            if (marks[i] > max) max = marks[i]; // Find Highest
            if (marks[i] < min) min = marks[i]; // Find Lowest
        }

        double average = (double) total / marks.length;
```

```
        // 5. Display Analytics
        System.out.println("\n--- CLASS SUMMARY ---");
        System.out.println("Total Marks: " + total);
        System.out.println("Class Average: " + average);
        System.out.println("Highest Score: " + max);
        System.out.println("Lowest Score: " + min);

        // 6. Search Functionality (Linear Search)
        System.out.println("\n--- SEARCH SYSTEM ---");
        System.out.print("Enter a score to search for: ");
        int target = scanner.nextInt();
        boolean found = false;

        for (int i = 0; i < marks.length; i++) {
            if (marks[i] == target) {
                System.out.println("Score " + target + " found at Student " + (i + 1));
                found = true;
                break; // Stop searching
            }
        }

        if (!found) {
            System.out.println("Score " + target + " does not exist in this class.");
        }

        System.out.println("==============================");
        scanner.close();
    }
}
```

```
=== STUDENT MARKS ANALYZER ===
How many students are in the class? 3

--- ENTER MARKS ---
Enter marks for Student 1: 85
Enter marks for Student 2: 92
Enter marks for Student 3: 78

--- CLASS SUMMARY ---
Total Marks: 255
```

```
Class Average: 85.0
Highest Score: 92
Lowest Score: 78

--- SEARCH SYSTEM ---
Enter a score to search for: 92
Score 92 found at Student 2
==============================
```

## Step-by-Step Execution:

1. **Dynamic Array Creation:** We ask the user for `numStudents` and create the `marks` array with that exact size (`new int[numStudents]`).
2. **Data Input Loop:** We use a `for` loop from 0 to `length-1` to pause and ask the user to input data for every single array slot.
3. **Analytics Loop:** A single loop calculates the sum, maximum, and minimum simultaneously. We initialize `max` and `min` with `marks[0]` to ensure negative/zero errors don't occur.
4. **Search Loop:** A standard Linear Search logic block looking for an exact match, using a boolean flag to track success.

---

IT STUDY HUB

# 8. Module Assessment and Interview Prep

Test your mastery of Data Handling, Arrays, and Strings to prepare for academia and industry interviews.

## A. Practice Questions (Test Your Logic)

1. Write a program to declare an integer array of size 5, assign values 10, 20, 30, 40, 50, and print the values using a `for` loop.
2. Write a Java program to calculate the average of all numbers stored in a `double[]` array.
3. Create a program that takes an array of ints and prints them in reverse order (start loop at `length - 1`).
4. Given an array `int[] arr = {4, 2, 8, 1, 9}`, write logic to find the smallest number.
5. Write a program using nested loops to add two 3x3 matrices.
6. Given `String s1 = "Hello";` and `String s2 = new String("Hello");`, write code to prove that `==` evaluates to false but `.equals()` evaluates to true.
7. Write a program to count how many vowels (a, e, i, o, u) exist in a given String. (Hint: Use `.charAt()` inside a loop).
8. Use `StringBuilder` to reverse the string 'IT Study Hub' and print the result.
9. Write a program that takes a String '100' and a String '50', parses them into integers using `Integer.parseInt()`, and prints their mathematical sum.
10. Write a program to check if a given String is a Palindrome (reads the same forwards and backwards).
11. Identify the error: `int[] numbers = new int[5]; numbers[5] = 10;`
12. Explain what output `System.out.println("Java".substring(1, 3));` will produce.
13. Write a Java program to find duplicate elements in a 1D array.
14. Explain the difference between `array.length` (property) and `string.length()` (method).
15. Write a program that replaces all spaces in a string with hyphens '-' using `.replace()`.

IT STUDY HUB

## B. University Viva Questions & Answers

### Q: What is an Array in Java?

**Ans:** An array is a data structure containing a fixed-size sequential collection of elements of the same data type.

### Q: Can you change the size of an array after it is created?

**Ans:** No, arrays are fixed-length in Java. Once initialized with a specific size, that size cannot be altered.

### Q: What is the index of the first and last element of an array?

**Ans:** The first element is at index 0. The last element is at index (length - 1).

### Q: What exception is thrown if you try to access an invalid array index?

**Ans:** ArrayIndexOutOfBoundsException.

### Q: What is a Multidimensional array?

**Ans:** An array of arrays. It represents data in a tabular format using rows and columns (e.g., int[][]).

### Q: What is Linear Search?

**Ans:** A basic searching algorithm that sequentially checks each element of an array until a match is found or the whole array has been searched.

### Q: Why is String considered an Object and not a primitive type?

**Ans:** Because it holds a sequence of characters and provides complex built-in methods (like .length(), .substring()) which primitive types do not possess.

### Q: What does it mean that Strings are 'Immutable'?

**Ans:** It means the contents of a String object cannot be modified after it is created. Any operation that alters the string actually creates a brand new String object in memory.

### Q: What is the String Pool?

**Ans:** A special storage area in Java heap memory. Java stores string literals here to reuse them and save memory. If two literals have the exact same text, they share the same memory reference in the pool.

### Q: What is the difference between == and .equals() when comparing Strings?

**Ans:** == compares object memory references (are they the exact same object in RAM?). .equals() compares the actual character contents of the strings.

IT STUDY HUB

**Q: When should you use StringBuilder instead of String?**

**Ans:** When you are performing many string modifications, especially inside loops. StringBuilder is mutable, preventing the creation of thousands of garbage strings in memory.

**Q: What is the difference between StringBuilder and StringBuffer?**

**Ans:** Both are mutable strings. StringBuffer is synchronized (thread-safe, slower), while StringBuilder is non-synchronized (not thread-safe, much faster).

**Q: What is a Wrapper class?**

**Ans:** A class that encapsulates (wraps) a primitive data type into an object. Examples include Integer for int, Double for double, and Character for char.

**Q: What is Autoboxing?**

**Ans:** The automatic conversion made by the Java compiler from a primitive type to its corresponding object wrapper class (e.g., int to Integer).

**Q: What is Unboxing?**

**Ans:** The automatic conversion from a wrapper class object back to its corresponding primitive type (e.g., Integer to int).

**Q: What method is used to convert a String representing a number into a primitive int?**

**Ans:** Integer.parseInt(String)

**Q: What exception occurs if Integer.parseInt() is given text that isn't a number?**

**Ans:** NumberFormatException.

**Q: How do you find the length of an array versus the length of a String?**

**Ans:** For an array, use the property 'array.length'. For a String, use the method 'string.length()'. Note the parentheses.

**Q: How does the .trim() method work?**

**Ans:** It removes all leading (front) and trailing (back) whitespace characters from a String.

**Q: What does the .substring(int beginIndex, int endIndex) method do?**

**Ans:** It extracts a portion of a string starting at beginIndex and stopping right before endIndex.

IT STUDY HUB

## C. Beginner Interview Questions

### Interview Q: Why did Java designers make the String class immutable?

**Ans:** For security, performance, and memory efficiency. Since String is heavily used in network connections, file paths, and database URLs, making it immutable prevents unauthorized modifications. It also allows the JVM to safely share strings via the String Pool, saving massive amounts of RAM.

### Interview Q: Explain the difference between creating a string using literals vs using the 'new' keyword.

**Ans:** String s = "Text" creates or reuses an object in the String Pool. String s = new String("Text") forces Java to create a brand new object in standard Heap memory, bypassing the optimization of the String Pool.

### Interview Q: How do you sort an array without writing your own sorting loop manually?

**Ans:** By using the built-in utility class Arrays. Calling Arrays.sort(arrayName) will automatically sort the array in ascending order using a highly optimized internal algorithm.

### Interview Q: If you do not initialize the elements of an integer array, what values do they hold?

**Ans:** Java automatically initializes them with default values. For an int array, every slot defaults to 0. For a boolean array, they default to false. For an object array (like String[]), they default to null.

### Interview Q: What is a NullPointerException?

**Ans:** It is a runtime error thrown when a program attempts to use an object reference that has not been initialized (points to 'null'). For example, trying to call .length() on a String that is currently null.

### Interview Q: Why can't we use primitive types (like int) in advanced Java Collections like ArrayList?

**Ans:** Java Collections are designed strictly to hold Objects. Therefore, we must use Wrapper classes (like Integer) which act as objects, allowing Collections to store, retrieve, and manage primitive data.

IT STUDY HUB

### Interview Q: What is the time complexity difference between searching an unsorted array vs a sorted array?

**Ans:** Searching an unsorted array requires Linear Search (O(n) time—checking every element). If the array is sorted, you can use Binary Search (O(log n) time), which halves the search area every step, making it exponentially faster for massive datasets.

### Interview Q: How does StringBuilder edit text without creating new objects?

**Ans:** Internally, StringBuilder uses a resizable primitive char[] array. When you append text, it modifies this internal array directly. If it runs out of space, it automatically creates a larger internal array and copies the data over, completely avoiding object recreation overhead.

### Interview Q: Write on the whiteboard: How do you swap two elements in an array at index i and j?

**Ans:** You need a temporary variable. int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;

### Interview Q: What is the meaning of public static void main(String[] args)? Focus on the args parameter.

**Ans:** args is a 1-Dimensional Array of Strings. It is used to capture command-line arguments. If a user runs the program from the terminal and types 'java ProgramName file.txt', the string 'file.txt' is stored at args[0].

---

IT STUDY HUB

# 10. Module Summary and Key Terms

## Key Terms Glossary

- **Contiguous:** Memory blocks that are placed directly next to each other physically.
- **Index:** The specific numerical position of an element within an array (always 0-indexed).
- **Traversal:** The process of systematically visiting every element in a data structure using a loop.
- **Immutable:** Unchangeable. Once an object is created, its internal state cannot be modified.
- **String Pool:** An optimized memory location where Java caches string literals to conserve RAM.
- **Autoboxing:** The automatic conversion from a primitive type to its Object Wrapper equivalent.

## Module Summary

In this module, you broke past the limitations of single variables. You learned how to declare, initialize, and traverse 1D and 2D **Arrays**, enabling you to manage hundreds of data points efficiently. You wrote algorithms to search through data and calculate large-scale metrics.

We then dissected the **String** class. You learned the critical difference between `==` and `.equals()`, manipulated text using built-in methods, and utilized `StringBuilder` for high-performance string editing. Finally, you bridged the gap

between primitives and objects using **Wrapper Classes**, culminating in the comprehensive Student Marks Analysis project.

## WHAT COMES NEXT?

### Module 5 — Object-Oriented Programming (OOP)

Until now, we have written "Procedural" code—a list of instructions run top-to-bottom. It's time to learn how software is built in the real world. In the next volume, you will master Object-Oriented Programming. You will learn to design your own custom Classes, instantiate Objects, and model real-world concepts using Encapsulation, Constructors, and the `this` keyword.

**IT STUDY HUB SERIES**